

Algorithms, AI & Data Science II

Prof. Dr. Ingo Scholtes

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Motivation

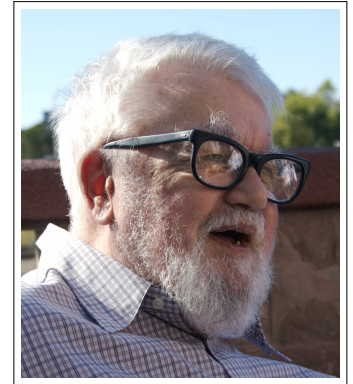
- ▶ central question in AI is how we can **represent knowledge about the world**
- ▶ we often use **natural language** to represent knowledge

examples

- ▶ “If it rains, I am wet.”
- ▶ “If it rains and I have an umbrella, I am not wet.”

questions

- ▶ can we use a **formal language** to describe knowledge about the world?
- ▶ can we use this formal language to **automatically reason** about the world? → L05 – L08



John Mc Carthy (1927 – 2011)
Turing Award 1971

image credit: Wikimedia user null0, CC-BY-SA

Notes:

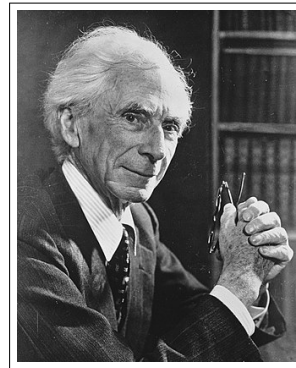
- **L05: Propositional Logic and Boolean Algebra** 14.05.2025
- **Educational objective:** We show how we can use formal logics to represent knowledge. We introduce propositional logic and Boolean algebra, introduce basic normal forms and explore logic optimization algorithms.
 - Logical Foundations of Computer Science and AI
 - Propositional Logic
 - Boolean Algebra and Normal Forms
 - Logic Optimization
- **Handout version:** 2025/05/13 17:48:08

Notes:

Introduction to formal logics

- ▶ a statement that can either be true or false is called a **proposition**, e.g. “It rains.” or “If it rains, then I am wet.”
- ▶ we call a proposition that can not be simplified further **atom**, e.g. “It rains.”
- ▶ we use **logical connectives** to create new propositions, e.g. “It rains and I have an umbrella.” or “If it rains, then I am happy.”

“The veracity of these formulas does not concern us. Propositional logic is not the study of truth, but of the **relationship between the truth of one statement and that of another.**”
→ S Hedman, 2004



Bertrand Russell, 1897 – 1970

image credit: Nationaal Archief, Fotocollectie Anefo, CC-0

- ▶ formal (mathematical) study of logics pioneered by **early 20th century philosophers**

Notes:

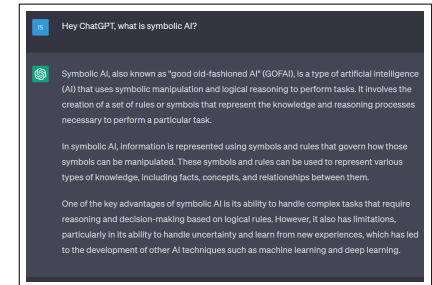
Symbolic AI vs. Machine Learning

- ▶ formal logics is basis for **knowledge representation, proof theory, and automated reasoning**
- ▶ basis for **symbolic artificial intelligence** using
 - ▶ symbolic representations of knowledge (i.e. formal language)
 - ▶ rules to manipulate those symbolic representations

“Symbolic artificial intelligence is the term for the collection of all methods in artificial intelligence research that are based on high-level symbolic (human-readable) representations of problems, logic and search. [...] Symbolic AI was the dominant paradigm of AI research from the mid-1950s until the middle 1990s.” → Wikipedia

- ▶ makes it difficult **to treat uncertainty** and incorporate **patterns in massive data sets** → L08 – L17

Notes:



ChatGPT answer to question
“What is symbolic AI”

image credit: openai.com, screenshot from 08/05/2023

Propositional logic

- **propositional logic** is a **context-free formal language** consisting of **formulas that either evaluate to true or false**

definition

- A **propositional variable** is an **atomic formula** that can either be true or false, e.g. X_0
- **True** and **False** are atomic formulas that are always true or false.
- We can use **logical connectives** $\wedge$ (AND, conjunction), $\vee$ (OR, disjunction), $\neg$ (NOT, negation), $\implies$ (if-then, implication), $\iff$ (iff, biconditional) to recursively create propositional formulas. If A and B are formulas, then $\neg A$, $A \wedge B$, $A \vee B$, $A \implies B$ and $A \iff B$ are formulas as well.
- We can use brackets to change evaluation order (or avoid ambiguity). I.e. if A is formula then (A) is a formula as well.

- mapping $(X_1, \dots, X_k) \leftarrow (\text{True}, \dots, \text{False})$ of propositional variables to truth values is called **assignment**

examples

- **True** is a (constant) atomic formula
- X_0 is atomic formula with propositional variable X_0
- $\neg X_0 \vee X_1$ is formula with propositional variables X_0 and X_1
- $((X_0 \wedge X_1) \implies X_2) \vee X_1$ is a propositional formula

question

how can we evaluate the logical value of a formula for a given assignment?

Truth tables

- **truth tables** define logical value of formulas for **any assignment of propositional variables**
- we can use them to define **semantics** of logical connectives
- we use 0 and 1 to represent logical values **False** and **True** → cf. slide 12

conjunction

X_1	X_0	$X_0 \wedge X_1$
0	0	0
0	1	0
1	0	0
1	1	1

disjunction

X_1	X_0	$X_0 \vee X_1$
0	0	0
0	1	1
1	0	1
1	1	1

negation

X_0	$\neg X_0$
0	1
1	0

implication

X_1	X_0	$X_0 \implies X_1$
0	0	1
0	1	0
1	0	1
1	1	1

biconditional

X_1	X_0	$X_0 \iff X_1$
0	0	1
0	1	0
1	0	0
1	1	1

Notes:

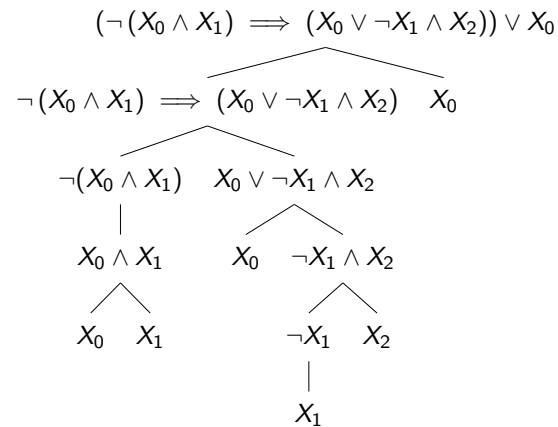
- propositional logic = Aussagenlogik = statement logics = propositional calculus

Notes:

- In a later part of today's lecture, we will see that the representation of truth values by the integer numbers 0 and 1 actually has some deeper implications. In particular, it allows us to map basic logical connectives to arithmetic operations, which is the basis for Boolean algebra → see slide 12 ff.

Recursive evaluation and operator precedence

- ▶ we can evaluate complex formula by **recursively applying semantics of logical connectives**
- ▶ we define **tree**, where nodes are (sub-)formula, links are logical connectives, and leafs are atoms
- ▶ order of evaluation defined by **operator precedence** $\neg, \wedge, \vee, \implies, \iff$
- ▶ we use brackets to change evaluation order (or avoid ambiguity)



example

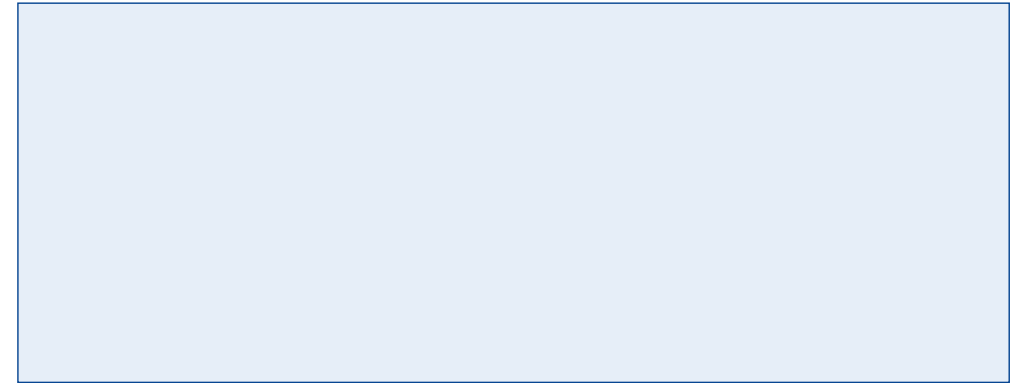
$$(\neg(X_0 \wedge X_1) \implies (X_0 \vee \neg X_1 \wedge X_2)) \vee X_0$$

Group exercise 05-01

1/2

Evaluate the following expression for the assignment $(X_0, X_1, X_2) \leftarrow (\text{True}, \text{False}, \text{True})$

$$\neg X_0 \wedge X_1 \implies X_0 \vee \neg X_1 \wedge X_2 \vee X_0$$



Notes:

Notes:

Group exercise 05-01

2/2

Evaluate the following expression for the assignment $(X_0, X_1, X_2) \leftarrow (\text{False}, \text{False}, \text{False})$

$$(\neg (X_0 \wedge X_1) \implies (X_0 \vee \neg X_1 \wedge X_2)) \vee X_0$$



Notes:

Tautologies and logical equivalence

definition

A formula that, independent from variable assignment, is always true is called **tautology**. A proposition that is always false is called **contradiction**.

► negations of tautologies are contradictions and vice-versa

► the following formulas are (useful) tautologies

- $X \implies (X \vee Y)$ addition
- $(X \wedge Y) \implies X$ conjunctive simplification
- $(X \implies Y) \iff (\neg X \vee Y)$
- $(X \iff Y) \iff ((X \implies Y) \wedge (Y \implies X))$

► two formulas X and Y are **logically equivalent** ($X \equiv Y$) iff $X \iff Y$ is a tautology, i.e.

- $X \implies Y \equiv \neg X \vee Y$
- $X \iff Y \equiv (X \implies Y) \wedge (Y \implies X)$

tautology $X \vee \neg X$

X	$\neg X$	$X \vee \neg X$
0	1	1
1	0	1

contradiction $X \wedge \neg X$

X	$\neg X$	$X \wedge \neg X$
0	1	0
1	0	0

$X \implies Y \equiv \neg X \vee Y$

X	Y	$X \implies Y$	$\neg X \vee Y$
0	0	1	1
0	1	1	1
1	0	0	0
1	1	1	1

Notes:

Tautology and contradiction rules

- ▶ the following equivalences hold **iff formula X is a tautology**

- ▶ $(X \wedge Y) \equiv Y$
- ▶ $(X \vee Y) \equiv X$

- ▶ the following equivalences hold **iff formula X is a contradiction**

- ▶ $(X \wedge Y) \equiv X$
- ▶ $(X \vee Y) \equiv Y$

- ▶ we can use logical equivalences to **transform and simplify formulas**

equivalences for $X \wedge Y$

X	Y	$X \wedge Y$
0	0	0
0	1	0
1	0	0
1	1	1

equivalences for $X \vee Y$

X	Y	$X \vee Y$
0	0	0
0	1	1
1	0	1
1	1	1

De Morgan's Law

- ▶ we can use negation to **transform disjunction (OR) into conjunction (AND)** (and vice-versa)

De Morgan's law

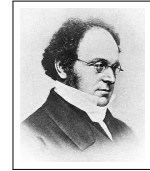
- ▶ **negation of a disjunction** is conjunction of negations, i.e.

$$\neg(X_1 \vee X_2) \equiv \neg X_1 \wedge \neg X_2$$

- ▶ **negation of a conjunction** is disjunction of negations, i.e.

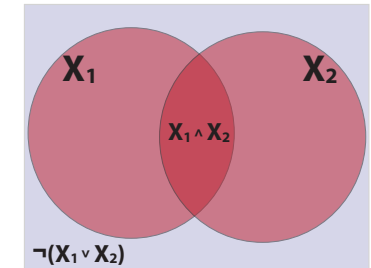
$$\neg(X_1 \wedge X_2) \equiv \neg X_1 \vee \neg X_2$$

- ▶ we can use **Venn diagram** for a visual proof



Auguste De Morgan (1806 – 1871)

image credit: Wikimedia Commons, Public domain

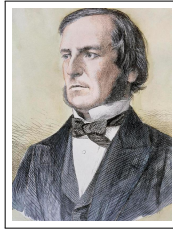


Notes:

Notes:

From Propositional Logic to Boolean Algebra

- ▶ formal treatment of logics pioneered by **George Boole**
- ▶ Boole introduced **algebraic perspective** on propositional logics in analogy to common arithmetics → G Boole, 1854
- ▶ basic logical operators can be mapped to **arithmetic operations on integers** $\mathbb{B} := \{0, 1\}$ representing False and True



George Boole (1815 – 1864)

image credit: Wikimedia Commons, public domain

basic operations in Boolean algebra

- ▶ $X_1 \wedge X_2 \equiv X_1 \cdot X_2 = \min(X_1, X_2)$
- ▶ $X_1 \vee X_2 \equiv X_1 + X_2 - X_1 \cdot X_2 = \max(X_1, X_2)$
- ▶ $\neg X_1 =: \bar{X}_1 \equiv 1 - X_1$
- ▶ $\implies, \iff$ can be expressed using $\wedge, \vee, \neg$
- ▶ propositional formula with n variables defines **Boolean function** $f : \mathbb{B}^n \rightarrow \mathbb{B}$

implication and biconditional

- ▶ $X_1 \implies X_2 \equiv \neg X_1 \vee X_2$
- ▶ $X_1 \iff X_2 \equiv (X_1 \wedge X_2) \vee (\neg X_1 \wedge \neg X_2)$

Properties of Boolean Algebra

1/2

- ▶ Boolean algebra satisfies some of the laws of arithmetic operations → exercise sheet 02

properties of Boolean algebra

1. $\wedge$ and $\vee$ are **commutative**, i.e.
 $X_0 \vee X_1 = X_1 \vee X_0$ and $X_0 \wedge X_1 = X_1 \wedge X_0$
2. $\wedge$ and $\vee$ are **associative**, i.e.
 $X_0 \wedge (X_1 \wedge X_2) = (X_0 \wedge X_1) \wedge X_2$ and
 $X_0 \vee (X_1 \vee X_2) = (X_0 \vee X_1) \vee X_2$
3. $\wedge$ is **distributive over** $\vee$, i.e.
 $X_0 \wedge (X_1 \vee X_2) = (X_0 \wedge X_1) \vee (X_0 \wedge X_2)$
4. **identity law** for $\vee$ and $\wedge$, i.e.
 $X_0 \vee \text{False} = X_0$ and $X_0 \wedge \text{True} = X_0$
5. **annulment law** for $\wedge$, i.e.
 $X_0 \wedge \text{False} = \text{False}$

identity laws

X_0	False	$X_0 \vee \text{False}$
0	0	0
1	0	1

X_0	True	$X_0 \wedge \text{True}$
0	1	0
1	1	1

annulment law

X_0	False	$X_0 \wedge \text{False}$
0	0	0
1	0	0

Notes:

Notes:

Properties of Boolean Algebra

2/2

- compared to arithmetic operations, Boolean algebra satisfies **additional laws** → Exercise sheet 02

additional properties of Boolean algebra

- annulment law for $\vee$** , i.e.
 $X_0 \vee \text{True} = \text{True}$
- idempotence of $\vee$ and $\wedge$** , i.e.
 $X_0 \vee X_0 = X_0$ and
 $X_0 \wedge X_0 = X_0$
- absorption with $\wedge$ and $\vee$** , i.e.
 $X_0 \wedge (X_0 \vee X_1) = X_0$ and
 $X_0 \vee (X_0 \wedge X_1) = X_0$
- $\vee$ is **distributive over $\wedge$** , i.e.
 $X_0 \vee (X_1 \wedge X_2) = (X_0 \vee X_1) \wedge (X_0 \vee X_2)$
- conjunction/disjunction with complement**
 $X_0 \vee \bar{X}_0 = \text{True}$ and $X_0 \wedge \bar{X}_0 = \text{False}$

annulment law

X_0	True	$X_0 \vee \text{True}$
0	1	1
1	1	1

absorption law

X_1	X_0	$X_0 \wedge (X_0 \vee X_1)$
0	0	0
0	1	1
1	0	0
1	1	1

Conjunctive and disjunctive normal forms

- formula is in **conjunctive normal form (CNF)** if it is a conjunction of disjunctions (also called **maxterms**)
- formula is in **disjunctive normal form (DNF)** if it is a disjunction of conjunctions (also called **minterms**)
- we can use De Morgan's law to bring CNF into DNF and vice-versa → Exercise sheet 02
- CNF or DNF is in **canonical normal form** iff for each min- or maxterm each variable occurs exactly once
- important basis for **logic optimization, satisfiability and logical inference** → L05 - Satisfiability and Logical Inference

example 1: CNF

$$(X_1 \vee \bar{X}_2 \vee X_3) \wedge (\bar{X}_1 \vee X_2 \vee X_3) \\ (X_1 + \bar{X}_2 + X_3)(\bar{X}_1 + X_2 + X_3)$$

example 2: DNF

$$(X_1 \wedge \bar{X}_2 \wedge X_3) \vee (\bar{X}_1 \wedge X_2) \\ (X_1 \bar{X}_2 X_3) + (\bar{X}_1 X_2)$$

example 3: CDNF

$$(X_1 \wedge \bar{X}_2 \wedge X_3) \vee (\bar{X}_1 \wedge X_2 \wedge \bar{X}_3) \vee (\bar{X}_1 \wedge X_2 \wedge X_3) \\ (X_1 \bar{X}_2 X_3) + (\bar{X}_1 X_2 \bar{X}_3) + (\bar{X}_1 X_2 X_3)$$

Notes:

Notes:

- Example 1 is in conjunctive normal form. Moreover it is in canonical normal form, as X_1, X_2, X_3 each appear exactly once in each maxterm.
- Example 2 is disjunctive normal form. However, it is not in canonical normal form. What can we do to bring it in canonical normal form? → self-study questions

Truth tables and normal forms

- ▶ we can use the truth table to bring any formula into canonical DNF or CNF
- ▶ we can specify formula as **sum of min-terms**, i.e. rows in truth table with **output one** (DNF)

$$f(X_0, X_1) = (\bar{X}_1 \wedge X_0) \vee (X_1 \wedge X_0) = m_1 + m_3$$

- ▶ equivalently, we can specify formula as **product of max-terms**, i.e. rows in truth table with **output zero** (CNF)

$$f(X_0, X_1) = (X_1 \vee X_0) \wedge (\bar{X}_1 \vee X_0) = M_0 \cdot M_2$$

- ▶ we use m_i and M_i to refer to the i -th min- and max-term, respectively
- ▶ min- and max-terms are called **clauses**

example

	X_1	X_0	$f(X_0, X_1)$
0	0	0	0
1	0	1	1
2	1	0	0
3	1	1	1

Group Exercise 05-02

1/2

Use a truth table to bring the following formula into **canonical disjunctive normal form**.

$$f(X_0, X_1, X_2) = (\bar{X}_1 \wedge X_0) \vee (X_2 \wedge X_0)$$

Notes:

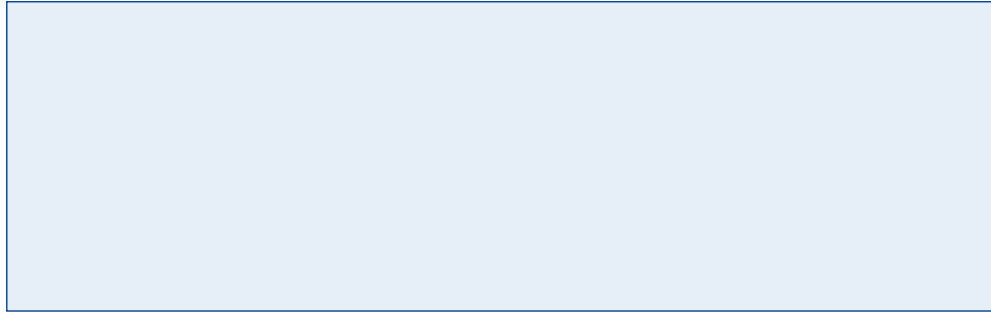
Notes:

Group Exercise 05-02

2/2

Use a truth table to bring the following formula into **canonical conjunctive normal form**.

$$f(X_0, X_1, X_2) = (\bar{X}_1 \wedge X_0) \vee (X_2 \wedge X_0)$$



Notes:

Logic optimization

- ▶ we often want to simplify a propositional formula with n variables
- ▶ for given formula F find simpler formula F' that is **logically equivalent**, i.e. $F \equiv F'$ for all assignments
- ▶ important problem in **digital logics**, as smaller circuits are cheaper, smaller and consume less power
- ▶ naive approach: choose CNF or DNF depending on whether there are more 1- or 0-rows in truth table

minimization of propositional formula is **NP-complete**, i.e. there is no known polynomial-time algorithm that always finds a minimal formula

example

the following two formulas are logically equivalent:

$$\begin{aligned} F := & (X_2 \vee X_1 \vee X_0) \wedge \\ & (X_2 \vee \bar{X}_1 \vee X_0) \wedge \\ & (X_2 \vee \bar{X}_1 \vee \bar{X}_0) \wedge \\ & (\bar{X}_2 \vee X_1 \vee X_0) \wedge \\ & (\bar{X}_2 \vee \bar{X}_1 \vee X_0) \end{aligned}$$

$$F' := (\bar{X}_1 \wedge X_0) \vee (X_2 \wedge X_0)$$

Notes:

Algebraic optimization

1/2

- ▶ we can apply laws of Boolean algebra to perform **algebraic optimization** of Boolean functions
- ▶ consider **Boolean function** with minterms $m_0, m_2, m_5, m_7, m_8, m_{10}, m_{13}, m_{15}$
- ▶ for brevity we use $X_i X_j$ to denote $X_i \wedge X_j$ and $X_i + X_j$ to denote $X_i \vee X_j$

$$f(X_0, X_1, X_2, X_3) = \bar{X}_3 \bar{X}_2 \bar{X}_1 \bar{X}_0 + \bar{X}_3 \bar{X}_2 X_1 \bar{X}_0 + \bar{X}_3 X_2 \bar{X}_1 X_0 + \bar{X}_3 X_2 X_1 X_0 + X_3 \bar{X}_2 \bar{X}_1 \bar{X}_0 + X_3 \bar{X}_2 X_1 \bar{X}_0 + X_3 X_2 \bar{X}_1 X_0 + X_3 X_2 X_1 X_0$$

- ▶ applying the **law of distributivity** we obtain

$$f(X_0, X_1, X_2, X_3) = \bar{X}_2 \bar{X}_0 (\bar{X}_3 \bar{X}_1 + \bar{X}_3 X_1 + X_3 \bar{X}_1 + X_3 X_1) + \bar{X}_3 X_2 \bar{X}_1 X_0 + \bar{X}_3 X_2 X_1 X_0 + X_3 X_2 \bar{X}_1 X_0 + X_3 X_2 X_1 X_0$$

- ▶ again applying the **law of distributivity** we obtain

$$f(X_0, X_1, X_2, X_3) = \bar{X}_2 \bar{X}_0 (\bar{X}_3 (\bar{X}_1 + X_1) + X_3 (\bar{X}_1 + X_1)) + \bar{X}_3 X_2 \bar{X}_1 X_0 + \bar{X}_3 X_2 X_1 X_0 + X_3 X_2 \bar{X}_1 X_0 + X_3 X_2 X_1 X_0$$

Notes:

Algebraic optimization

2/2

- ▶ using the **complement, identity and annulment laws** we obtain

$$f(X_0, X_1, X_2, X_3) = \bar{X}_2 \bar{X}_0 + \bar{X}_3 X_2 \bar{X}_1 X_0 + \bar{X}_3 X_2 X_1 X_0 + X_3 X_2 \bar{X}_1 X_0 + X_3 X_2 X_1 X_0$$

- ▶ we again apply the **law of distributivity** (twice)

$$\begin{aligned} f(X_0, X_1, X_2, X_3) &= \bar{X}_2 \bar{X}_0 + X_2 X_0 (\bar{X}_3 \bar{X}_1 + \bar{X}_3 X_1 + X_3 \bar{X}_1 + X_3 X_1) \\ &= \bar{X}_2 \bar{X}_0 + X_2 X_0 (\bar{X}_3 (\bar{X}_1 + X_1) + X_3 (\bar{X}_1 + X_1)) \end{aligned}$$

- ▶ using the **complement, identity and annulment laws** we obtain a minimal DNF that is logically equivalent to the original expression

$$f(X_0, X_1, X_2, X_3) = \bar{X}_2 \bar{X}_0 + X_2 X_0$$

Notes:

Karnaugh-Veitch maps

- ▶ for formulas with up to four variables, we can use **graphical minimization approach**
- ▶ for n variables, create **two-dimensional grid with 2^n cells**, each cell corresponding to row in truth table
- ▶ label rows/columns according to **Gray code of variable assignments**, i.e. assignments of neighbouring rows/columns differ in exactly one variable
- ▶ consider DNF with minterms that correspond to **maximally large rectangular groups of ones** with areas that are power of two

X_2	X_1	X_0	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

		$X_1 X_0$			
		00	01	11	10
X_2	0	0	1	1	0
	1	0	1	0	0

example

$f(X_0, X_1, X_2) = X_0\bar{X}_2 + X_0\bar{X}_1$

- ▶ due to Gray code, we can **interpret grid as torus**

Group Exercise 05-03

Use a Karnaugh-Veitch map to minimize the Boolean function f given in the truth table below.

X_3	X_2	X_1	X_0	f
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



Notes:

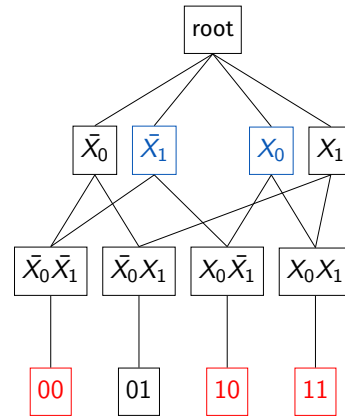
Notes:

Quine-McCluskey algorithm

- graphical approach of Karnaugh-Veitch maps is intuitive for humans but **difficult for more than four variables**
- consider **search graph of possible variable assignments**, where bottom-most nodes represent all possible minterms
- any Boolean function in DNF is defined by **subset S of bottom-most nodes**, e.g.

$$\bar{X}_0\bar{X}_1 + X_0\bar{X}_1 + X_0X_1$$

- idea: find **minimal set of nodes that covers S**
- starting from minterms, **iteratively combine terms that differ in exactly one position**

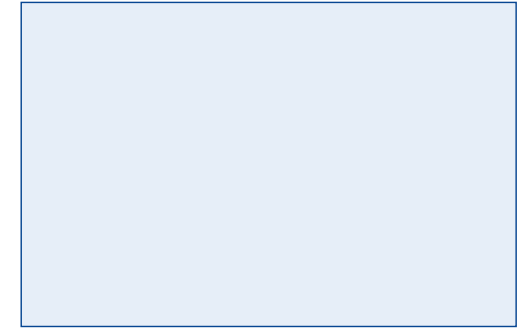


Group Exercise 05-04

1/2

We use the Quine-McCluskey algorithm to minimize function f given in the truth table below.

X_3	X_2	X_1	X_0	f
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



Notes:

- Logic Minimization is an NP-complete problem, i.e. there is no known polynomial time algorithm. The time complexity of the Quine-McCluskey algorithm grows exponentially with the number of variables, i.e. it is useful only for relatively small inputs. For larger numbers of variables we thus have to resort to heuristic logic minimization algorithms.

Notes:

Group Exercise 05-04

2/2



Notes:

Practice Session

- ▶ we implement logical connectives in python
- ▶ we use pandas to generate truth tables
- ▶ we use an implementation of the Quine-McCluskey algorithm to minimize Boolean functions

```
03-02 Logic Minimization
May 3 2023
Ingo Scholtes

from quine_mcccluskey import QuineMcCluskey

qm = QuineMcCluskey()
qm.simplify(ones=[1,5,7])

('1-01', '1-1')

qm = QuineMcCluskey()
qm.simplify(ones=[0,2,5,7,8,10,11,15])

('1-0-0', '1-1-1')
```

practice session

see notebooks 05-01 and 05-02 in **gitlab** repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

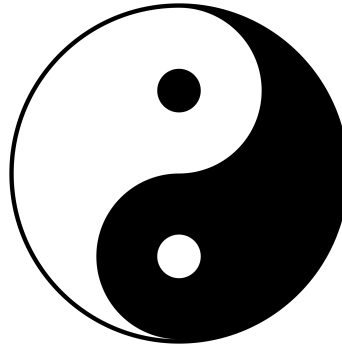
Notes:

In summary ...

- ▶ we introduced **propositional logic and Boolean algebra**
- ▶ context-free **formal language** to represent propositions that can either be **true or false**
- ▶ we considered **truth tables, normal forms and logic optimization**
- ▶ propositional logic is important foundation for **symbolic AI**
- ▶ basis for **more expressive logic systems** like first-order and higher-order logic → L06 and L07

open questions

1. how can we **prove theorems** in propositional logic?
2. can we automate this process?



Ying Yang

image credit: Wikimedia commons, public domain

Self-study questions

1. Consider the formula shown on slide 6. Which of the brackets change the evaluation result?
2. Use a truth table to prove the two distributivity laws of $\wedge$ and $\vee$ in Boolean algebra.
3. Give an example that shows that one of the distributivity laws is violated for ordinary arithmetic operations $\cdot$ and $+$ on integer numbers.
4. Use a truth table to prove the two absorption properties of Boolean algebra.
5. Give an example that shows that none of the absorption laws holds for ordinary arithmetic operations $\cdot$ and $+$ on integer numbers.
6. Consider a truth table for a Boolean algebra expression in n variables, where the expression gives rise to m ones in the table. Under which condition is it more reasonable to use CNF or DNF to represent the expression?
7. Use a truth table to prove De Morgan's law.
8. Prove the equivalence $X_1 \iff X_2 \equiv (X_1 \wedge X_2) \vee (\neg X_1 \wedge \neg X_2)$
9. Consider the expression $(X_1 \wedge \neg X_2 \wedge X_3) \vee (\neg X_1 \wedge X_2)$ in CNF. Give a logically equivalent Boolean expression in canonical CNF.
10. Use Karnaugh-Veitch maps to minimize the formula consisting of minterms $m_2 + m_3 + m_7 + m_8$.

Notes:

Notes:

References

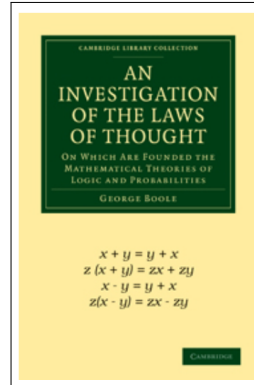
reading list

- ▶ G Boole: **An Investigation of the Laws of Thought on Which are Founded the Mathematical Theories of Logic and Probabilities**, 1854
- ▶ M Karnaugh: **The Map Method for Synthesis of Combinational Logic Circuits**, Transactions of the American Institute of Electrical Engineers, 72(5), 1953
- ▶ EJ McCluskey Jr.: **Minimization of Boolean Functions**, Bell System Technical Journal, 1956
- ▶ J McCarthy: **Programs with Common Sense**, Conference on the Mechanization of Thought Processes, 1959
- ▶ A Newell: **The knowledge level**, Artificial Intelligence 18(1), 1982
- ▶ B Nebel: **Logics for Knowledge Representation**, Int. Encyc. Social and Behavioral Sciences, 2001

acknowledgments

lecture slides partly based on the following books and lectures:

- ▶ S Hedman: **A First Course in Logic**, Oxford Texts in Logic, 2004
- ▶ U Schöning: **Logik für Informatiker**, Spektrum Akadem. Verlag, 2000
- ▶ C Meinel, M Mundhenk: **Mathematische Grundlagen der Informatik**, Springer, 2014
- ▶ H Strass: **Foundations of Knowledge Representation**, TU Dresden



Notes: